ERHVERVSAKADEMI
AARHUS



Programmering modul 04

CSIK E2026

ERHVERVSAKADEMI
AARHUS

I dag

Kommandoer igen

Datastrukturer

Arrays og strings

5 + 20 + 5 minutters pause undervejs

Slutter 16:00

Kommandoer igen

```
write e
```

```
eval e
```

```
let v = e in
  s
end let
```

```
set v = e
```

```
if e then
  s1
else
  s2
end if
```

```
if e then
  s
end if
```

```
while e do
  s
end while
```

```
skip
```

```
s1
s2
⋮
sn
```

Kommandoer igen

```
write e
```

```
code(e)
wrt
```

```
eval e
```

```
code(e)
pop
```

```
let v = e in
  s
end let
```

```
code(e)
code(s)
pop
```

```
set v = e
```

```
code(e)
save n
```

```
if e then
  s1
else
  s2
end if
```

```
code(e)
jmpf else
code(s1)
jump done
else:
code(s2)
done:
```

```
if e then
  s
end if
```

```
code(e)
jmpf done
code(s)
done:
```

```
while e do
  s
end while
```

```
loop:
code(e)
jmpf done
code(s)
jump loop
done:
```

```
skip
```

```
s1
s2
⋮
sn
```

```
code(s1)
code(s2)
⋮
code(sn)
```

Men er det ikke bare et legetøjssprog?

```
write e
```

```
eval e
```

```
let v = e in
  s
end let
```

```
set v = e
```

```
if e then
  s1
else
  s2
end if
```

```
if e then
  s
end if
```

```
while e do
  s
end while
```

```
skip
```

```
s1
s2
⋮
sn
```

Men er det ikke bare et legetøjssprog?

Det er i hvert fald et modelsprog

Designet til undervisningsbrug

- tilstræbt 1-1 og eksplicit sammenhæng mellem begreber og syntaks
- fokus på læring

Simplere end industrielle sprog (JavaScript, C#, Python, C, C++, Java, Go, Rust, Dart, ...)

- fokus på produktivitet
- fokus på fortsat relevans

ERHVERVSAKADEMI
AARHUS

Men er det ikke bare et legetøjssprog?

Hvis det er simplere end industrielle sprog – er der så noget, vi ikke kan bruge det til?

Er der fx noget, vi ikke kan beregne?

Og som man kan beregne med et rigtigt programmeringssprog?

ERHVERVSAKADEMI
AARHUS

Men er det ikke bare et legetøjssprog? Nej



Alan Turing (1912-1954)

Formaliserede algoritmebegrebet og beregnelighed

Brød nazisternes Enigma kryptosystem under 2. verdenskrig

(se filmen "The Imitation Game" fra 2014)

Vores simple programmeringssprog er **Turing-komplet**

= det kan bruges til at beregne alt det, der kan beregnes*

= det, som kan beregnes med JavaScript, C#, Python, C, C++, Java, Go, Rust, Dart, ...*

= det, som kan beregnes med en **Turing-maskine**

= det, som kan beregnes med lambda-kalkulen

= det, som kan beregnes med partielle rekursive funktioner

= det, som kan beregnes med registermaskiner og tæller-maskiner

= det, som kan beregnes med Post systemer

= det, som kan beregnes med cellulære automater

= det, som kan beregnes med endelige automater med to stakke

* kræver ubegrænset arbejdslager

Kommandoer igen: hvor kommer styrken fra?

```
write e
```

```
eval e
```

```
let v = e in
  s
end let
```

```
set v = e
```

```
if e then
  s1
else
  s2
end if
```

```
if e then
  s
end if
```

```
while e do
  s
end while
```

```
skip
```

```
s1
s2
⋮
sn
```

Kommandoer igen: hvor kommer styrken fra?

```
write e
```

```
eval e
```

```
let v = e in
  s
end let
```

```
set v = e
```

```
if e then
  s1
else
  s2
end if
```

```
if e then
  s
end if
```

```
while e do
  s
end while
```

```
skip
```

```
s1
s2
⋮
sn
```

Vi kan bygge
vilkårligt komplekse
kommandoer
af simple kommandoer,
hierarkisk

ERHVERVSAKADEMI
AARHUS

Byg en kommando...

s

ERHVERVSAKADEMI
AARHUS

Byg en kommando...

```
s
s
s
```

```
s1
s2
:
sn
```

Byg en kommando...

```
let v = e in
  s
end let
s
s
```

```
let v = e in
  s
end let
```

Byg en kommando...

```
let v = e in
  if e then
    s
  end if
end let
s
s
```

```
if e then
  s
end if
```

Byg en kommando...

```
let v = e in
  if e then
    while e do
      s
    end while
  end if
end let
s
s
```

```
while e do
  s
end while
```

Byg en kommando...

```
let v = e in
  if e then
    while e do
      s
      s
    end while
  end if
end let
s
s
```

```
s1
s2
⋮
sn
```

Byg en kommando...

```
let v = e in
  if e then
    while e do
      write e
      s
    end while
  end if
end let
s
s
```

```
write e
```

Byg en kommando...

```
let v = e in
  if e then
    while e do
      write e
      set v = e
    end while
  end if
end let
s
s
```

```
set v = e
```

Byg en kommando...

```
let v = e in
  if e then
    while e do
      write e
      set v = e
    end while
  end if
end let
if e then
  s
else
  s
end if
s
```

```
if e then
  s1
else
  s2
end if
```

Byg en kommando...

```
let v = e in
  if e then
    while e do
      write e
      set v = e
    end while
  end if
end let
if e then
  let v = e in
    s
  end let
else
  s
end if
s
```

```
let v = e in
  s
end let
```

Byg en kommando...

```
let v = e in
  if e then
    while e do
      write e
      set v = e
    end while
  end if
end let
if e then
  let v = e in
    write e
  end let
else
  write e
end if
write e
```

```
write e
```

Byg en kommando...

```
let a = 42 in
  if a > 0 then
    while a < 100 do
      write a * a
      set a = a + 1
    end while
  end if
end let
if 7 < 5 + 3 then
  let b = 8 * 42 in
    write b
  end let
else
  write 9 = 3 + 6 and 9 <> 3 * 6
end if
write 99
```

ERHVERVSAKADEMI
AARHUS

Din tur

ERHVERVSAKADEMI
AARHUS

Din tur

Byg en kommando

- Det er lige meget, hvad den gør
- Den skal kunne afvikles i codelab'et "Imperative programming" (fx på side 2)
- Den skal have mindst fire niveauer af indlejring / indryk
- Den skal være mindst 20 linjer lang

ERHVERVSAKADEMI
AARHUS

Din tur

Byg en kommando

- Det er lige meget, hvad den gør
- Den skal kunne afvikles i codelab'et "Imperative programming" (fx på side 2)
- Den skal have mindst fire niveauer af indlejring / indryk
- Den skal være mindst 20 linjer lang

Klik på <reset, så din makker ikke kan se outputtet

Med skuldermakker:

- Forudsig, hvad hinandens kommandoer gør: Skriv det forventede output/fejl ned
- Kør kommandoerne og sammenlign med jeres forudsigelser

Pause 13:15-13:20

ERHVERVSAKADEMI
AARHUS



ERHVERVSAKADEMI
AARHUS

Datastrukturer

ERHVERVSAKADEMI
AARHUS

Datastrukturer

- En disciplineret måde at organisere data på
- … sådan at vi kan opbygge og aflæse data effektivt og på systematisk vis
- Nogle datastrukturer kan også opdateres

Datastrukturer

- En disciplineret måde at organisere data på
- … sådan at vi kan opbygge og aflæse data effektivt og på systematisk vis
- Nogle datastrukturer kan også opdateres

Vigtigste datastrukturer:

- træ = hierarchy = tree
- vektor = list = array = string = sequence
- ordbog = map = dictionary

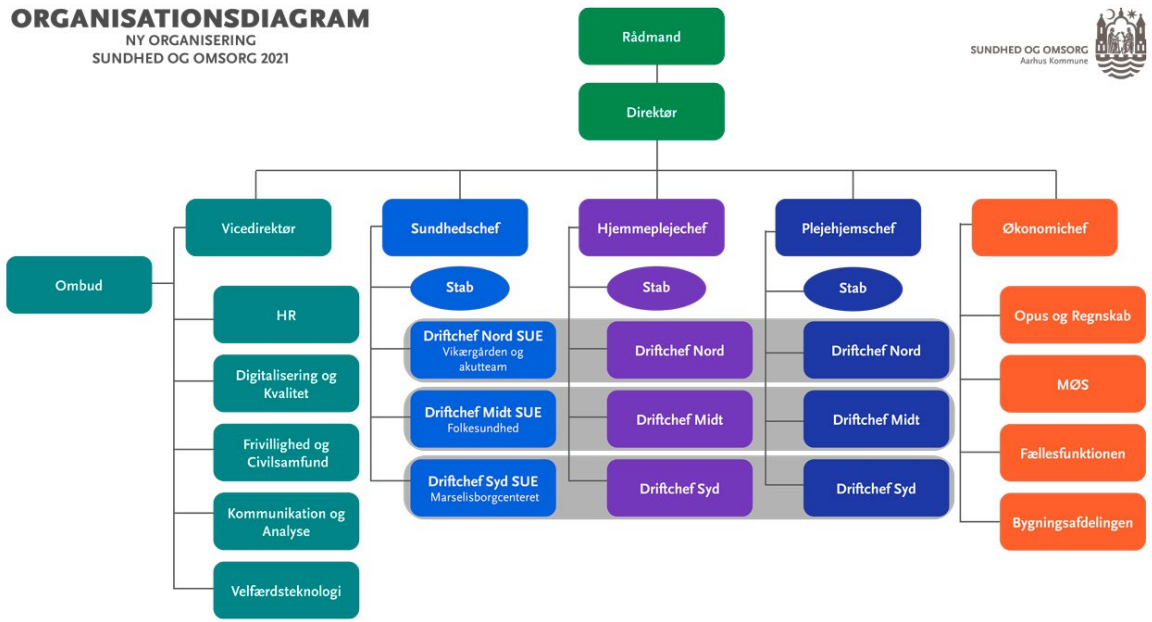
Træ / Hierarchy / Tree

- Hierarkisk struktur af knuder. Der er én rod-knude uden forældre.
- Alle øvrige knuder har én forælder. Knuder kan indeholde værdier.
- Knuder uden børn kaldes "blade". Knuder med børn kaldes "indre knuder".

ERHVERVSAKADEMI
AARHUS

Træ / Hierarchy / Tree

- Hierarkisk struktur af knuder. Der er én rod-knude uden forældre.
- Alle øvrige knuder har én forælder. Knuder kan indeholde værdier.
- Knuder uden børn kaldes "blade". Knuder med børn kaldes "indre knuder".

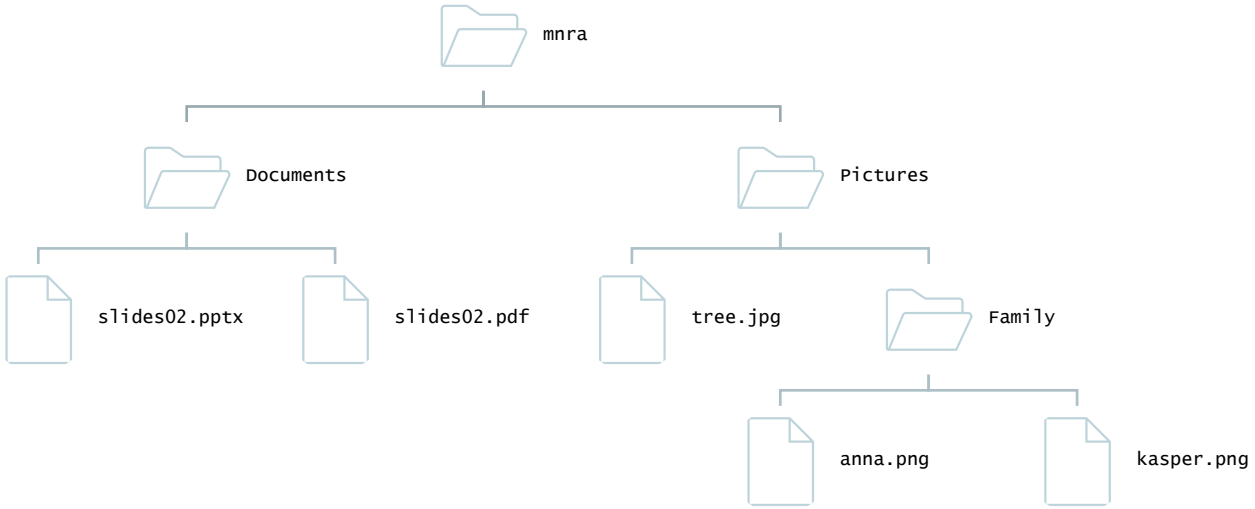


it-folk vender gerne træer på hovedet 🙃 – roden øverst, bladene nederst

ERHVERVSAKADEMI
AARHUS

Træ / Hierarchy / Tree

- Hierarkisk struktur af knuder. Der er én rod-knude uden forældre.
- Alle øvrige knuder har én forælder. Knuder kan indeholde værdier.
- Knuder uden børn kaldes "blade". Knuder med børn kaldes "indre knuder".



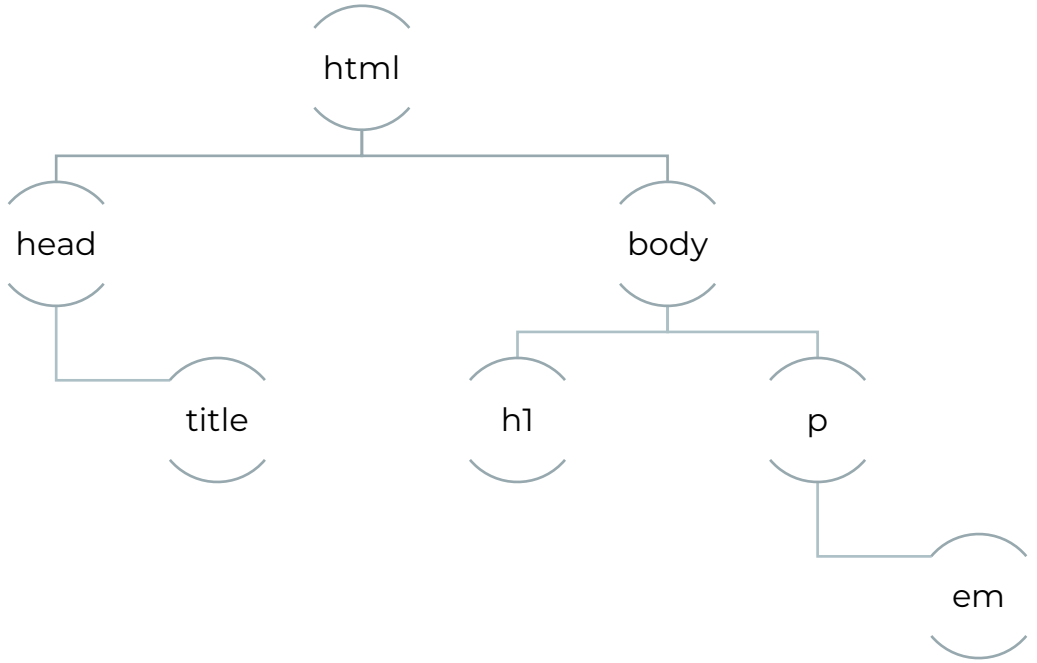
it-folk vender gerne træer på hovedet 🙃 – roden øverst, bladene nederst

ERHVERVSAKADEMI
AARHUS

Træ / Hierarchy / Tree

- Hierarkisk struktur af knuder. Der er én rod-knude uden forældre.
- Alle øvrige knuder har én forælder. Knuder kan indeholde værdier.
- Knuder uden børn kaldes "blade". Knuder med børn kaldes "indre knuder".

```
<html>
  <head>
    <title>My Homepage</title>
  </head>
  <body>
    <h1>Welcome to my home on the web</h1>
    <p>
      This is a paragraph of <em>very</em> important text.
      Read it or be left in the dark!
    </p>
  </body>
</html>
```



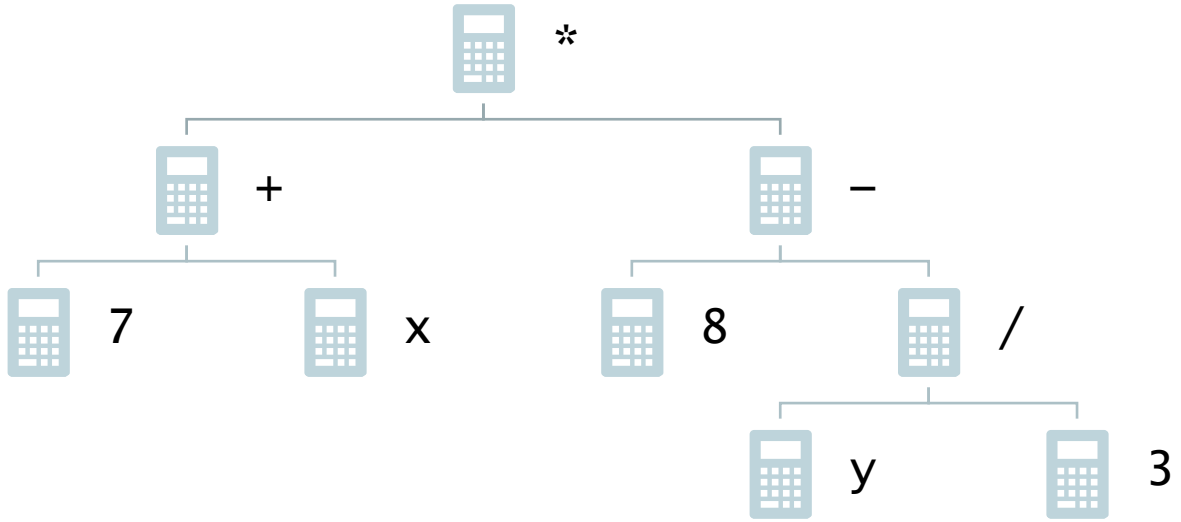
it-folk vender gerne træer på hovedet 🙃 – roden øverst, bladene nederst

ERHVERVSAKADEMI
AARHUS

Træ / Hierarchy / Tree

- Hierarkisk struktur af knuder. Der er én rod-knude uden forældre.
- Alle øvrige knuder har én forælder. Knuder kan indeholde værdier.
- Knuder uden børn kaldes "blade". Knuder med børn kaldes "indre knuder".

```
(7 + x) * (8 - y / 3)
```



it-folk vender gerne træer på hovedet 🙃 – roden øverst, bladene nederst

Vektor / List / Array / String / Sequence

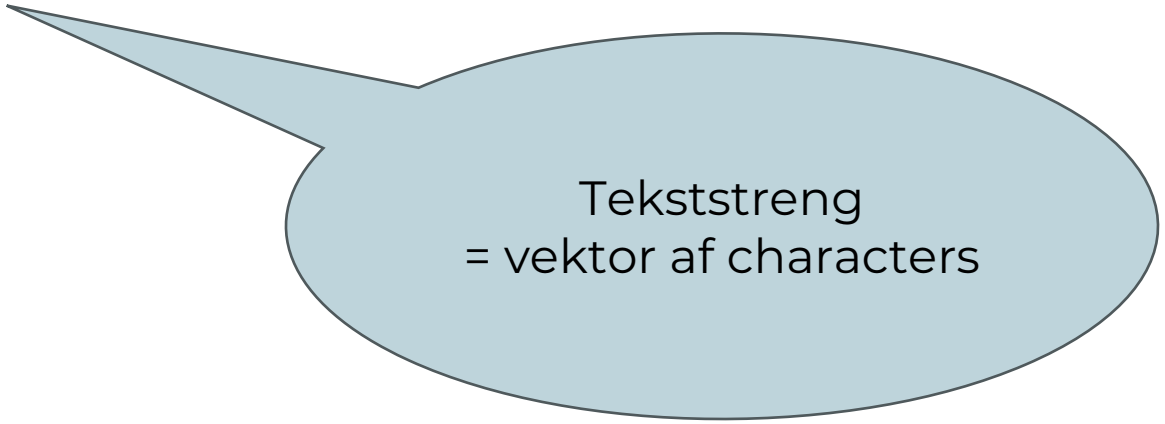
- Lineær struktur af *n* data-elementer
- Man kan slå elementer op per indeks
- Indices er de naturlige tal fra 0 (inklusiv) til *n* (eksklusiv)

ERHVERVSAKADEMI
AARHUS

Vektor / List / Array / String / Sequence

- Lineær struktur af *n* data-elementer
- Man kan slå elementer op per indeks
- Indices er de naturlige tal fra 0 (inklusiv) til *n* (eksklusiv)

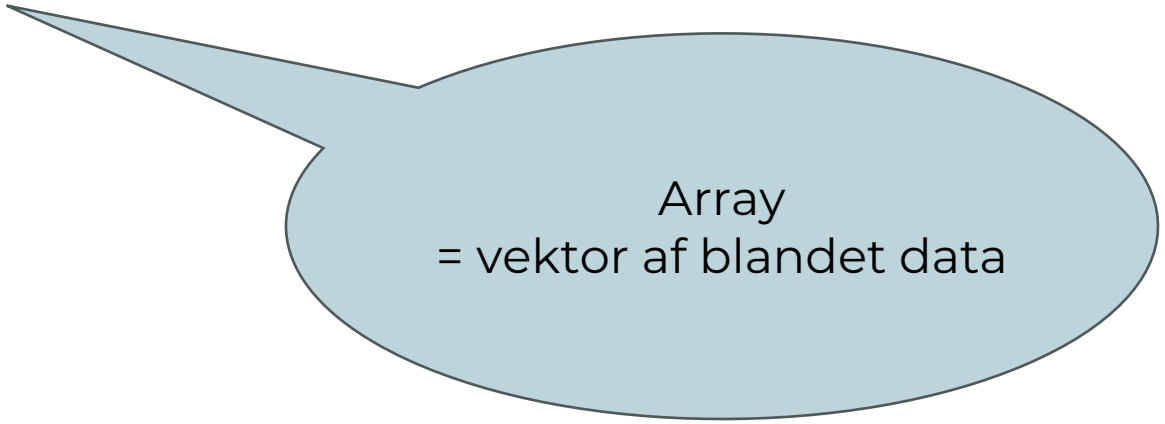
`s`	`'hello'`
`s[0]`	`'h'`
`s[1]`	`'e'`
`s[2]`	`'l'`
`s[3]`	`'l'`
`s[4]`	`'o'`
`s[5]`	fejl



Vektor / List / Array / String / Sequence

- Lineær struktur af *n* data-elementer
- Man kan slå elementer op per indeks
- Indices er de naturlige tal fra 0 (inklusiv) til *n* (eksklusiv)

`a`	`[42, true, 'hello', 'world']`
`a[0]`	`42`
`a[1]`	`true`
`a[2]`	`'hello'`
`a[3]`	`'world'`
`a[4]`	fejl



ERHVERVSAKADEMI
AARHUS

Ordbog / Map / Dictionary

- Samling af nøgle/værdi-par
- Nøgler er unikke
- Man kan slå værdier op per nøgle

ERHVERVSAKADEMI
AARHUS

Ordbog / Map / Dictionary

- Samling af nøgle/værdi-par
- Nøgler er unikke
- Man kan slå værdier op per nøgle

```
d               { 'dk': 'Copenhagen', 'no': 'Oslo', 'se': 'Stockholm' }

d['dk']         'Copenhagen'
d['no']         'Oslo'
d['se']         'Stockholm'
d['gb']         fejl
```

Indlejring / Nesting

- Datastrukturer kan indlejres i hinanden
- En vektor kan fx indeholde en anden vektor eller en ordbog

```
a               [42, true, 'hello', [7, 'world', {'dk': 25, 'se': 12}]]

a[0]            42
a[3]            [7, 'world', {'dk': 25, 'se': 12}]
a[3][0]         7
a[3][1]         'world'
a[3][1][2]      'r'
a[3][2]         {'dk': 25, 'se': 12}
a[3][2]['dk']   25
```

Hvad blev der af træerne?

Vi har syntaks for vektorer

Vi har syntaks for ordbøger

Hvilken syntaks kan vi bruge til at danne et træ?

Indlejring / Nesting

- Datastrukturer kan indlejres i hinanden
- En vektor kan fx indeholde en anden vektor eller en ordbog

```
a               [42, true, 'hello', [7, 'world', {'dk': 25, 'se': 12}]]

a[0]            42
a[3]            [7, 'world', {'dk': 25, 'se': 12}]
a[3][0]         7
a[3][1]         'world'
a[3][1][2]      'r'
a[3][2]         {'dk': 25, 'se': 12}
a[3][2]['dk']   25
```



ERHVERVSAKADEMI
AARHUS

Authentication log

På Hvedebro Maskinfabrik er IT-afdelingen langt om længe begyndt at opsamle data om, hvem der forsøger at logge ind på hvilke maskiner

Fætter kalder det en *auth log*

Hvad kunne en auth log indeholde og hvordan kunne de data være struktureret?

ERHVERVSAKADEMI
AARHUS

Codelabs

Følg slagplanen fra samarbejdsstrukturer04.pdf på Canvas

Tag jer god tid, så I får læring ud af det. I når ikke det hele nu.

Pause 14:05-14:25

ERHVERVSAKADEMI
AARHUS



ERHVERVSAKADEMI
AARHUS

Om arrays i "Imperative programming"

Om arrays i "Imperative programming"

```
[42, true, [87, nil]]
```

I "Arrays and dictionaries" så vi, hvordan man kan danne et array

Om arrays i "Imperative programming"

```
[42, true, [87, nil]]
```

I "Arrays and dictionaries" så vi, hvordan man kan danne et array

```
let a = [42, true, [87, nil]] in
  // do something with a
end let
```

Det kan man også i "Imperative programming"...

Om arrays i "Imperative programming"

```
[42, true, [87, nil]]
```

I "Arrays and dictionaries" så vi, hvordan man kan danne et array

```
let a = [42, true, [87, nil]] in
  // do something with a
end let
```

Det kan man også i "Imperative programming"...

```
let a = array[3] in
  set a[0] = 42
  set a[1] = true
  set a[2] = [87, nil]
  // do something with a
end let
```

... men her er det en forkortelse for noget mere grundlæggende

Om arrays i "Imperative programming"

```
[42, true, [87, nil]]
```

I "Arrays and dictionaries" så vi, hvordan man kan danne et array

```
let a = [42, true, [87, nil]] in
  // do something with a
end let
```

Det kan man også i "Imperative programming"...

```
let a = array[3] in
  set a[0] = 42
  set a[1] = true
  set a[2] = [87, nil]
  // do something with a
end let
```

... men her er det en forkortelse for noget mere grundlæggende

Hvad er det?
Hvad er et array en vektor af i "Imperative programming" (og Python)?

Array = vektor af variabler

- Lineær struktur af *n* variabler
- Man kan slå den enkelte variabel op per indeks
- Indices er de naturlige tal fra 0 (inklusiv) til *n* (eksklusiv)
- Længden af et array er fastlagt ved dannelsen, men de *n* variabler kan ændres senere

```
let a = [42, true, [87, nil]] in … end let
```

```
write len a // 3
```

```
write a[0]    // 42
write a[1]    // true
write a[2]    // [87, nil]
write a[2][0] // 87
write a[3]    // fejl
```

```
set a[1] = false
set a[2][1] = 88
write a // [42, false, [87, 88]]
```

Om arrays i "Imperative programming"

Syntaks:

```
array[e]
```

array constructor udtryk

Semantik:

1. Evaluér *e*
   Lad os sige, resultatet er *n*
2. Opret et array af længde *n* i arbejdslageret
3. Indsæt værdien **`nil`** på alle pladser i det nye array
4. Værdien af hele udtrykket er en reference til det nye array

```
[e1,…,en]
```

array literal udtryk

1. Evaluér $e_1,\ldots,e_n$ i den rækkefølge
   Lad os sige, resultatet er værdierne $v_1,\ldots,v_n$
2. Opret et array af længde *n* i arbejdslageret
3. Kopiér værdierne $v_1,\ldots,v_n$ ind i det nye array
4. Værdien af hele udtrykket er en reference til det nye array

ERHVERVSAKADEMI
AARHUS

Om arrays i "Imperative programming"

Syntaks:

```
array[e]
```

array constructor udtryk

Semantik:

1. Evaluér *e*
   Lad os sige, resultatet er *n*
2. Opret et array af længde *n* i arbejdslageret
3. Indsæt værdien **`nil`** på alle pladser i det nye array
4. Værdien af hele udtrykket er en reference til det nye array

Sideeffekt

```
[e1,…,en]
```

array literal udtryk

1. Evaluér $e_1,\ldots,e_n$ i den rækkefølge
   Lad os sige, resultatet er værdierne $v_1,\ldots,v_n$
2. Opret et array af længde *n* i arbejdslageret
3. Kopiér værdierne $v_1,\ldots,v_n$ ind i det nye array
4. Værdien af hele udtrykket er en reference til det nye array

Sideeffekt

ERHVERVSAKADEMI
AARHUS

Om arrays i "Imperative programming"

Syntaks:

Semantik:

```
e1[e2]
```

array indexing udtryk

1. Evaluér e_1
   Lad os sige, resultatet er en reference til et array af længde *n*
2. Evaluér e_2
   Lad os sige, resultatet er et heltal *idx*, hvor $0 \leq idx < n$
3. Værdien af hele udtrykket er den værdi, der er gemt på indeks *idx* i arrayet

```
set e1[e2] = e3
```

array indexing udtryk
på venstresiden i set-kommando

1. Evaluér e_1
   Lad os sige, resultatet er en reference til et array af længde *n*
2. Evaluér e_2
   Lad os sige, resultatet er et heltal *idx*, hvor $0 \leq idx < n$
3. Evaluér e_3
   Lad os sige, resultatet er værdien *v*
4. Overskriv elementet på indeks *idx* i arrayet med *v*

```
len e
```

array length udtryk

1. Evaluér *e*
   Lad os sige, resultatet er en reference til et array af længde *n*
2. Værdien af hele udtrykket er *n*

Array references

Hvordan forklarer vi programmets opførsel?

```
let a = array[3] in
let b = a in
  set b[2] = 99
  write a[2]
end let
```

Array references

Hvordan forklarer vi programmets opførsel?

```
let a = array[3] in
let b = a in
  set b[2] = 99
  write a[2]
end let
```

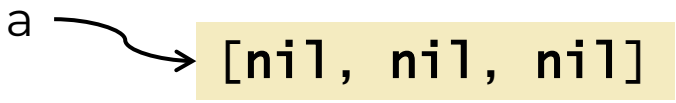
Prøv det selv
på en tilfældig side i
"Imperative Programming"
(fx side 2)

Hvad gør programmet?
Hvorfor det?

Array references

Hvordan forklarer vi programmets opførsel?

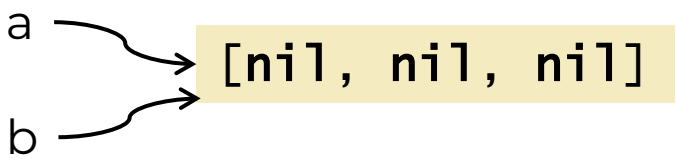
```
let a = array[3] in
let b = a in
  set b[2] = 99
  write a[2]
end let
```



Array references

Hvordan forklarer vi programmets opførsel?

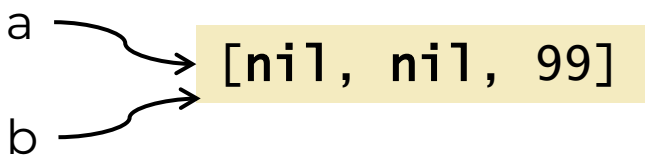
```
let a = array[3] in
let b = a in
  set b[2] = 99
  write a[2]
end let
```



Array references

Hvordan forklarer vi programmets opførsel?

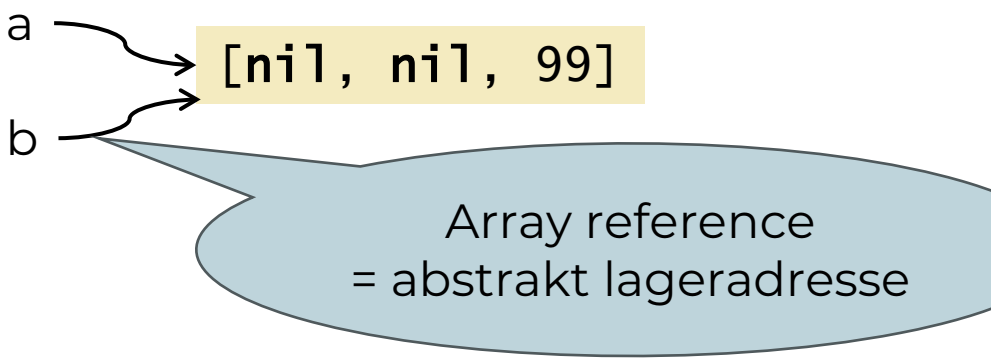
```
let a = array[3] in
let b = a in
  set b[2] = 99
  write a[2]
end let
```



Array references

Hvordan forklarer vi programmets opførsel?

```
let a = array[3] in
let b = a in
  set b[2] = 99
  write a[2]
end let
```

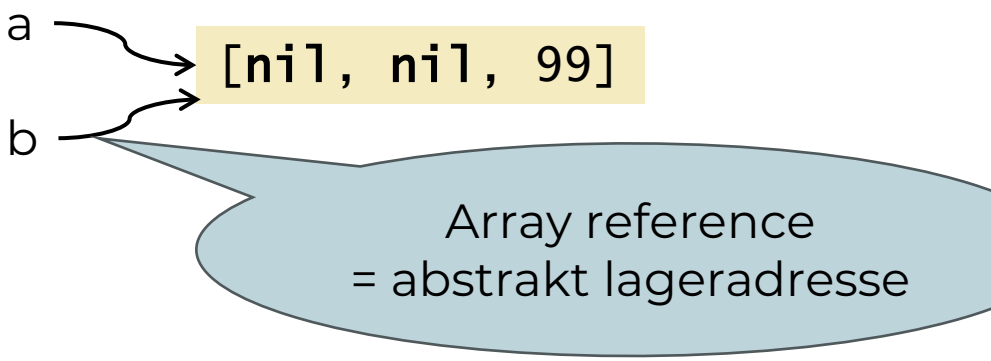


- Fem slags værdier: **`nil`**, sandhedsværdier, heltal, tekststrenge, array referencer
- Værdier er uforanderlige, når først de er dannet
- Enhver variabel indeholder præcist én værdi
- En variabel kan sættes til at indeholde en anden værdi med en **`set`**-kommando

Array references

Hvordan forklarer vi programmets opførsel?

```
let a = array[3] in
let b = a in
  set b[2] = 99
  write a[2]
end let
```



- Fem slags værdier: **`nil`**, sandhedsværdier, heltal, tekststrenge, array referencer
- Værdier er uforanderlige, når først de er dannet
- Enhver variabel indeholder præcist én værdi
- En variabel kan sættes til at indeholde en anden værdi med en **`set`**-kommando
- Arrays er *ikke* værdier (for arrays kan ændres)
- Variabler indeholder *ikke* arrays (for variabler indeholder kun værdier)

ERHVERVSAKADEMI
AARHUS

Codelabs

Følg slagplanen fra samarbejdsstrukturer04.pdf på Canvas

Spring til "Imperative programming" side 75

Tag jer god tid, så I får læring ud af det. I når ikke det hele nu.

Pause 15:10-15:15

ERHVERVSAKADEMI
AARHUS



ERHVERVSAKADEMI
AARHUS

Om strings i "Imperative programming"

Om strings i "Imperative programming"

Uformelt: tekst er en sekvens af skrifttegn

Formelt:

Tekststreng / string	= uforanderlig vektor af Unicode codepoints
Character	= tekststreng af længde 1
Glyf / glyph	= den grafiske form af et skrifttegn (afhænger af skriftsnit / font)

Om strings i "Imperative programming"

Uformelt: tekst er en sekvens af skrifttegn

Formelt:

Tekststreng / string	= uforanderlig vektor af Unicode codepoints
Character	= tekststreng af længde 1
Glyf / glyph	= den grafiske form af et skrifttegn (afhænger af skriftsnit / font)

Eksempel:

- Tre glyffer: a *a* *a*
- Samme character, `'a'`
- Unicode navn: "Latin Small Letter A"
- Unicode code point: 97

Om strings i "Imperative programming"

Syntaks:

Semantik:

```
ch e
```

code point conversion udtryk

1. Evaluér *e*
   Lad os sige, resultatet er et Unicode code point
   dvs. et heltal *n* med $0 \leq n \leq 1114111$
2. Værdien af hele udtrykket er den tilsvarende character
   dvs. en tekststreng af længde 1

```
cp e
```

code point conversion udtryk

1. Evaluér *e*
   Lad os sige, resultatet er en character
   dvs. en tekststreng af længde 1
2. Værdien af hele udtrykket er det tilsvarende Unicode code point
   dvs. et heltal *n* med $0 \leq n \leq 1114111$

Om strings i "Imperative programming"

Syntaks:

```
e1[e2]
```

string indexing udtryk

Semantik:

1. Evaluér e_1
   Lad os sige, resultatet er en tekststreng af længde *n*
2. Evaluér e_2
   Lad os sige, resultatet er et heltal *idx*, hvor $0 \leq idx < n$
3. Værdien af hele udtrykket er den character, der befinder sig på indeks *idx* i tekststrengen

```
len e
```

string length udtryk

1. Evaluér *e*
   Lad os sige, resultatet er en tekststreng af længde *n*
2. Værdien af hele udtrykket er *n*

ERHVERVSAKADEMI
AARHUS

Om strings i "Imperative programming"

Syntaks:

```
'abc{e1}def{e2}ghi{e3}jkl'
```

string interpolation udtryk

Semantik:

1. Evaluér e_1, e_2, e_3 i den rækkefølge
   Lad os sige, resultatet er tekststrengene s_1, s_2, s_3
2. Værdien af hele udtrykket er sammensætningen
   `'abc'` s_1 `'def'` s_2 `'ghi'` s_3 `'jkl'`

```
e1 join e2
```

join udtryk

1. Evaluér e_1
   Lad os sige, resultatet er et array af længde n
   og at elementerne kan konverteres til tekststrengene $s_1,\ldots,s_n$
2. Evaluér e_2
   Lad os sige, resultatet er tekststrengen s
3. Værdien af hele udtrykket er sammensætningen
   s_1 s s_2 s … s s_n

ERHVERVSAKADEMI
AARHUS

Codelabs

Følg slagplanen fra samarbejdsstrukturer04.pdf på Canvas

Tag jer god tid, så I får læring ud af det. I når ikke det hele nu.

Opsamling 15:50

ERHVERVSAKADEMI
AARHUS

Opsamling

Datastrukturer

Arrays og strings

Hvad var svært at forstå?

ERHVERVSAKADEMI
AARHUS

Opfølgende hjemmearbejde

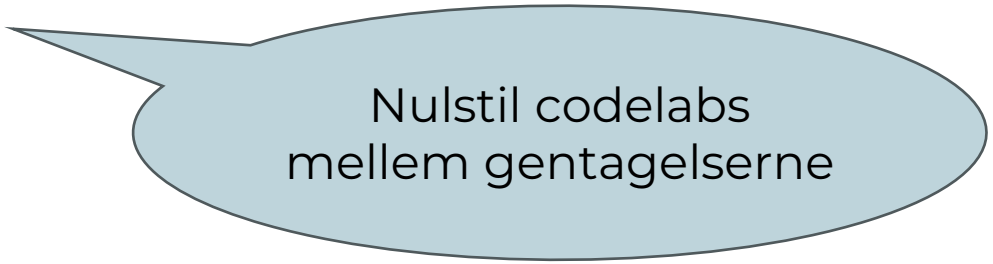
Gennemgå "Arrays and dictionaries" side 1-30 flere gange

Gennemgå "Imperative programming" side 75-84 og 89-100 flere gange

Det er øvelsesbaner, ikke afleveringer

Bliv ved, indtil

- teksten føles velkendt og let forståelig
- du kan løse opgaverne uden hjælp



Brug din studiegruppe så meget som muligt

- brug de nye begreber mundtligt: spørg, forklar, eksemplificér
- vær konkret og specifik

ERHVERVSAKADEMI
AARHUS

Opfølgende hjemmearbejde

Gennemgå "Arrays and dictionaries" side 1-30 flere gange

Gennemgå "Imperative programming" side 75-84 og 89-100 flere gange

Det er øvelsesbaner, ikke afleveringer

Bliv ved, indtil

- teksten føles velkendt og let forståelig
- du kan løse opgaverne uden hjælp

Brug din studiegruppe så meget som muligt

- brug de nye begreber mundtligt: spørg, forklar, eksemplificér
- vær konkret og specifik

Hjernen skal have
tid og gentagelser
for at lære effektivt

Tommelfingerregel: 10/24/7

ERHVERVSAKADEMI
AARHUS

Reaktioner på i dag

Spørgsmål?

Bekymringer?

Protester?

Kritik?